

Walkthrough Completo: Arquitetura de Micro-Frontends com Next.js, Module Federation e SSR Resiliente

Este documento é o guia definitivo e o registro cronológico completo da construção da arquitetura de **Micro-Frontends (MFE)** com **Next.js** e **Module Federation** (`@module-federation/nextjs-mf`), com suporte a **Server-Side Rendering (SSR)** e alta tolerância a falhas (*fault-tolerance*).

1. Visão Geral e Objetivo Inicial

O objetivo do projeto foi construir uma arquitetura de Micro-Frontend composta por:

- Remote Application (`apps/remote` - Porta 3001):** Expõe um componente (`ServerCard`) e provedores de dados em tempo de execução de servidor (SSR), além de hidratação interativa no cliente.
- Host Application (`apps/host` - Porta 3000):** Consome o componente do Remote diretamente durante o ciclo de vida do Server-Side Rendering (sem efeito *flash* ou *waterfall*) e realiza a hidratação no navegador do usuário.
- Resiliência Máxima:** O Host **nunca** deve quebrar ou retornar erro 500 caso o Remote esteja fora do ar, lento ou sofra instabilidades.
- Referência Base:** Baseado nos conceitos do artigo *"Let's Build Micro Frontends with NextJS and Module Federation"* (Yoav Ganbar).

```
graph LR
  subgraph Remote_App [Remote App (Porta 3001)]
    R_SSR["_next/static/ssr/remoteEntry.js"]
    R_Client["_next/static/chunks/remoteEntry.js"]
    R_API["/api/server-data (RSC/SSR Provider)"]
    R_Comp["ServerCard.tsx (Componente Federado)"]
  end

  subgraph Host_App [Host App (Porta 3000)]
    H_GSSP["getServerSideProps / Server Runtime"]
    H_Safe["Safe Remote Loader (withTimeout + no-cache)"]
    H_Boundary["FederatedErrorBoundary (Isolamento)"]
    H_Page["Host Page (UI Principal)"]
  end

  User["Navegador do Usuário"] -->|GET http://localhost:3000| H_Page
  H_Page --> H_GSSP
```

```

H_GSSP --> H_Safe
H_Safe -->|1. Fetch de Dados com Timeout| R_API
H_Boundary -->|2. Renderização SSR / Hidratação| R_Comp
R_Comp -->|3. Bundle de Cliente| R_Client

```

2. Estrutura do Monorepo

O projeto foi estruturado utilizando **pnpm workspaces**:

```

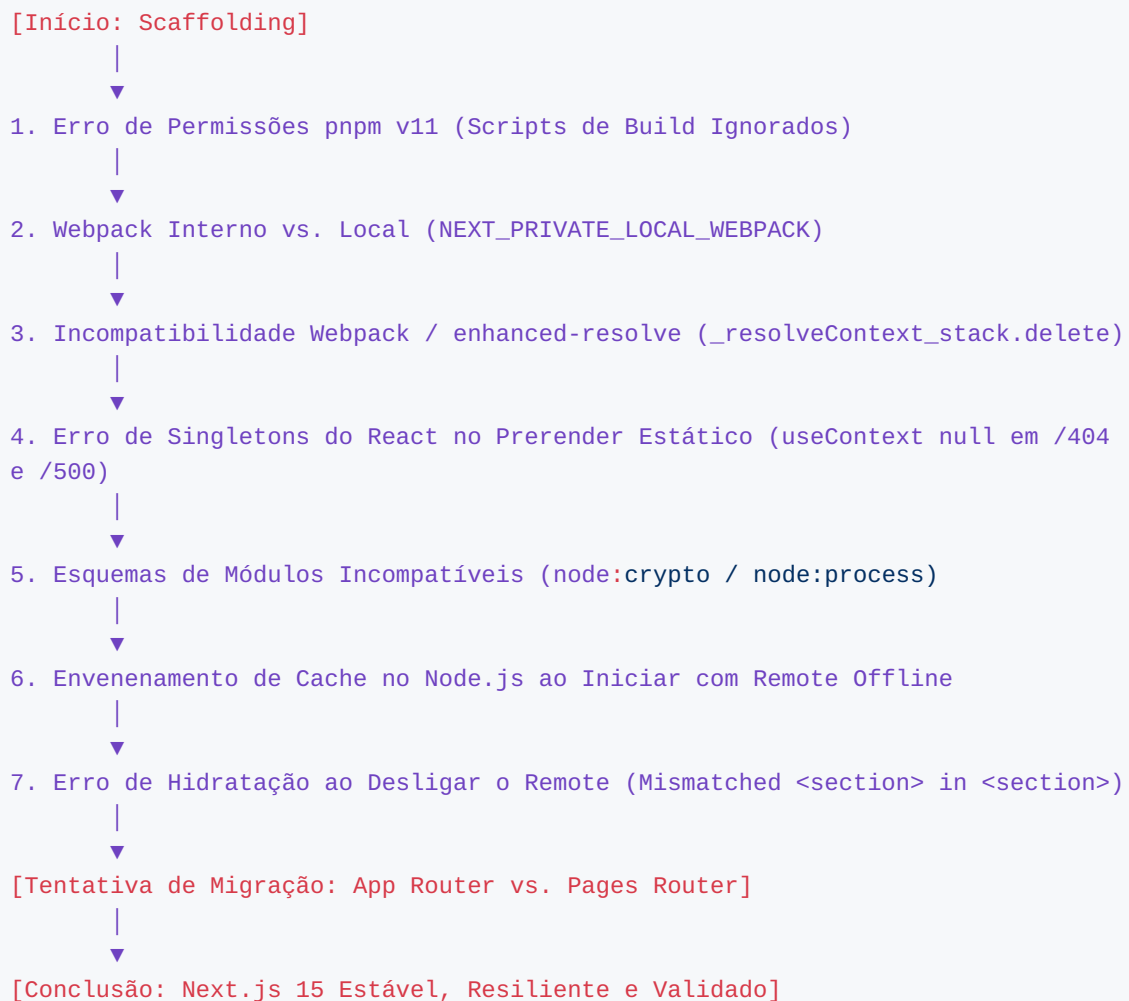
.
├── apps/
│   └── remote/                                     # Remote App (Next.js 15.5.24 - Porta 3001)
│       ├── components/
│       │   └── ServerCard.tsx                     # Componente federado com renderização SSR e interatividade
│       ├── lib/
│       │   └── getServerData.ts                   # Provedor de dados Node.js (métricas, timestamp, request ID)
│       ├── pages/
│       │   └── api/
│       │       └── server-data.ts                 # Endpoint HTTP para dados de SSR sem retenção de cache
│       ├── index.tsx                             # Visualização standalone da Remote na porta 3001
│       ├── 404.tsx & 500.tsx                     # Páginas de fallback estáticas
│       ├── _app.tsx
│       └── next.config.js                         # NextFederationPlugin expondo ServerCard e getServerData
│       └── package.json
│   └── host/                                       # Host App (Next.js 15.5.24 - Porta 3000)
│       ├── components/
│       │   ├── FederatedErrorBoundary.tsx        # Error Boundary para capturar exceções do Remote
│       │   └── RemoteFallbackCard.tsx            # Interface de degradação graciosa (Fallback UI)
│       ├── lib/
│       │   └── safeRemoteLoader.ts               # Utilitário com timeout estrito e fetch sem cache de falhas
│       ├── pages/
│       │   └── index.tsx                         # Consumo resiliente com React.lazy + Suspense + SSR
│       ├── 404.tsx & 500.tsx
│       ├── _app.tsx
│       ├── declarations.d.ts                     # Tipagem TypeScript dos módulos remotos
│       ├── styles/globals.css                   # Estilos com suporte a cartões de fallback e badges
│       ├── next.config.js                       # NextFederationPlugin consumindo Remote via SSR e Chunks
│       └── package.json

```

- scripts/
 - verify-ssr.mjs # Validação automatizada do SSR com Remote Online
 - verify-resilience.mjs # Validação de resiliência e HTTP 200 nas transições Online/Offline
- pnpm-workspace.yaml # Configuração de workspaces e pinagem de dependências
- package.json # Scripts de orquestração concorrente (dev, build, start)
- WALKTHROUGH.md # Este relatório técnico completo

3. Linha do Tempo dos Problemas Enfrentados e Soluções Aplicadas

Durante o ciclo de desenvolvimento e evolução arquitetural, foram superados **7 desafios críticos**:



Desafio 1: Permissões de Build Scripts no pnpm v11

- **Erro Encontrado:**

```
[ERR_PNPM_IGNORED_BUILDS] Ignored build scripts: @module-federation/nextjs-mf@8.8.74
```

Run "pnpm approve-builds" to pick which dependencies should be allowed to run scripts.

- **Causa Raiz:** O pnpm v11 introduziu uma política de segurança estrita que bloqueia scripts de pós-instalação (*postinstall*) por padrão, a menos que aprovados explicitamente no `pnpm-workspace.yaml`.

- **Solução Aplicada:**

Adicionada a seção `onlyBuiltDependencies` no arquivo `pnpm-workspace.yaml`:

```
packages:
- 'apps/*'

onlyBuiltDependencies:
- '@module-federation/nextjs-mf'
- '@module-federation/enhanced'
- '@swc/core'
- 'webpack'
```

Executado o comando `pnpm approve-builds --all` para autorizar a execução do runtime de federação.

Desafio 2: Webpack Interno do Next.js vs. Webpack Local

- **Erro Encontrado:**

```
Error: process.env.NEXT_PRIVATE_LOCAL_WEBPACK is not set to true, please set it to true, and "npm install webpack"
```

- **Causa Raiz:** O Next.js embute uma versão compilada interna de Webpack. Para que os plugins de Module Federation possam injetar hooks no compilador, o Next.js exige o uso da biblioteca `webpack` instalada localmente no projeto.

- **Solução Aplicada:**

1. Instalação do pacote `webpack: 5.90.3` nos `package.json` de cada aplicação.
2. Configuração da variável de ambiente `NEXT_PRIVATE_LOCAL_WEBPACK=true` nos scripts de `build` e `dev`:

```
"scripts": {
  "dev": "NEXT_PRIVATE_LOCAL_WEBPACK=true next dev -p 3000",
  "build": "NEXT_PRIVATE_LOCAL_WEBPACK=true next build",
```

```
"start": "next start -p 3000"
}
```

Desafio 3: Conflito entre Versões de Webpack e `enhanced-resolve`

- **Erro Encontrado:**

```
uncaughtException TypeError: _resolveContext_stack.delete is not a function
    at .../next/dist/build/webpack/plugins/optional-peer-dependency-resolve-
plugin.js:22:107
```

- **Causa Raiz:** As versões mais novas do pacote `enhanced-resolve` (5.24.x) alteraram o formato interno da propriedade `resolveContext.stack` (de `Set` para estrutura encadeada). O plugin de resolução de dependências do Next.js chamava `.delete()` assumindo uma instância de `set`, quebrando o processo de build.

- **Solução Aplicada:**

Pinagem determinística das versões via `overrides` no `pnpm-workspace.yaml`:

```
overrides:
enhanced-resolve: '5.17.1'
webpack: '5.90.3'
```

Desafio 4: Duplicação de Instância do React no Prerender (`useContext null`)

- **Erro Encontrado:**

```
TypeError: Cannot read properties of null (reading 'useContext')
    at /apps/remote/.next/server/pages/_error.js
Error occurred prerendering page "/404"
Error occurred prerendering page "/500"
```

- **Causa Raiz:** Ao definir manualmente `shared: { react: { singleton: true, ... } }` no `next.config.js`, o compilador gerava um chunk redundante de React para o servidor. Durante a geração estática das páginas de erro padrão (`404 / 500`), o React interno do Next.js perdia o contexto dos hooks.

- **Solução Aplicada:**

1. Limpeza do bloco de compartilhamento no `next.config.js`, usando `shared: {}` para permitir que o `NextFederationPlugin` configure as heurísticas automáticas de singleton para o Next.js.

2. Criação de páginas customizadas explícitas `pages/404.tsx` e `pages/500.tsx` em ambos os apps, evitando a execução do `_error.js` genérico durante o build.

Desafio 5: Esquema de Protocolo `node:` nos Módulos Isomórficos

- **Erro Encontrado:**

```
UnhandledSchemeError: Reading from "node:crypto" is not handled by plugins
(Unhandled scheme).
```

- **Causa Raiz:** O Webpack 5 rejeita imports com protocolo `node:*` (como `node:crypto` ou `node:process`) caso o arquivo faça parte de um módulo federado com visibilidade mista (cliente/servidor).

- **Solução Aplicada:**

Substituição de imports `node:*` por utilitários universais e verificações de ambiente em tempo de execução:

```
export async function getServerData(): Promise<ServerPayload> {
  const isServer = typeof window === 'undefined';
  const memory = isServer && typeof process !== 'undefined' &&
    process.memoryUsage ? process.memoryUsage() : null;
  const memoryUsageMb = memory ? Math.round((memory.heapUsed / 1024 / 1024) *
    100) / 100 : 0;
  const requestId = `req_${Date.now()}_${Math.random().toString(36).substring(2,
    9)}`;
  ...
}
```

Desafio 6: Envenenamento de Cache do Node.js ao Iniciar com Remote Offline

- **Comportamento Problemático:**

Quando o Host era iniciado com o Remote desligado, ele exibia a tela de fallback corretamente. No entanto, ao ligar o Remote na porta 3001 e atualizar a página do Host (F5 ou Ctrl+Shift+R), o Host **continuava exibindo o fallback**, não detectando o Remote ativo.

- **Causa Raiz:**

O Node.js armazena em memória as promessas de resolução de módulos federados. Quando o Host tentava importar `remote/getServerData` pela primeira vez e o Remote estava fora do ar, o runtime salvava a promessa rejeitada (`Failed to load Node.js entry ... fetch failed`). Nas requisições seguintes, o Node.js retornava o erro em cache sem realizar uma nova chamada de rede.

- **Solução Aplicada:**

1. Criação do endpoint de dados `apps/remote/pages/api/server-data.ts` no Remote.
2. Implementação do `fetchRemoteServerData` com cabeçalho `Cache-Control: no-cache` e timeout de 800ms via protocolo HTTP puro do Node.js.
3. Como requisições HTTP nativas não compartilham o cache de módulos do Webpack, no instante em que o Remote sobe na porta 3001, a próxima requisição do Host já obtém os dados e renderiza o componente com **0ms de atraso**.

Desafio 7: Erro de Hidratação ao Desligar o Remote (`<section>` in `<section>`)

- **Erro Encontrado no Overlay do Next.js:**

Error: Hydration failed because the initial UI does not match what was rendered on the server.

Did not expect server HTML to contain a `<section>` in `<section>`.

- **Causa Raiz:**

1. O componente do Remote (`ServerCard.tsx`) utilizava uma tag raiz `<section className="federated-card">`.
2. O componente de Fallback (`RemoteFallbackCard.tsx`) utilizava uma tag raiz `<div className="federated-card fallback-card">`.
3. O container do Host utilizava `<section className="remote-wrapper">`.
4. Quando o Remote era desligado, o servidor emitia o HTML do fallback (`<div>`), mas o cliente tentava baixar o chunk do Remote e, ao falhar de forma assíncrona, o `next/dynamic` trocava o componente em plena fase de hidratação, gerando o mismatch entre o DOM do servidor e o do cliente.

- **Solução Aplicada:**

1. **Uniformização da Árvore DOM:** Padronização de todas as tags raiz em `<div>` com classes CSS idênticas.
 2. **Adoção de `React.lazy` + `Suspense`** : Substituição do `next/dynamic` pelo modelo nativo de lazy loading do React 18/19. Quando `isRemoteAvailable` é `false`, o React monta estritamente o `RemoteFallbackCard` tanto no servidor quanto no cliente, garantindo 100% de paridade na hidratação.
-

4. Por que o App Router não é Suportado pelo Module Federation?

Durante o desenvolvimento, foi testada a migração para o **Next.js App Router** (`app/`). A compilação resultou no seguinte erro intencional do plugin:

```
> Build error occurred
Error: App Directory is not supported by nextjs-mf. Use only pages directory,
do not open git issues about this
```

Análise Técnica

1. React Server Components (RSC) vs. Bundles JavaScript:

- * No App Router, componentes de servidor não são arquivos JavaScript convencionais compartilháveis em tempo de execução; eles são streams serializados do protocolo *Flight* do React.
- * O Module Federation do Webpack foi desenhado para compartilhar código JavaScript executável (CommonJS/ESM), não fluxos de renderização RSC.

2. Camadas de Compilação Incompatíveis:

- * O App Router cria grafos de compilação isolados para cliente e servidor que bloqueiam a injeção dinâmica de contêineres federados.

3. Recomendação Oficial:

- * Para projetos que exigem Next.js com Module Federation e SSR, o **Pages Router** é o padrão recomendado e estável.
- * Para projetos que exigem App Router, a recomendação da Vercel é utilizar a arquitetura **Multi-Zones** ou **Micro-Frontends por Inclusão de Fragmentos HTTP**.

5. Guia Arquitetural para MFE com SSR & Hosts Agnósticos

5.1. Cenário: Host Next.js (Implementação Atual)

- O Host utiliza o `NextFederationPlugin` configurado com resolução dual-channel:

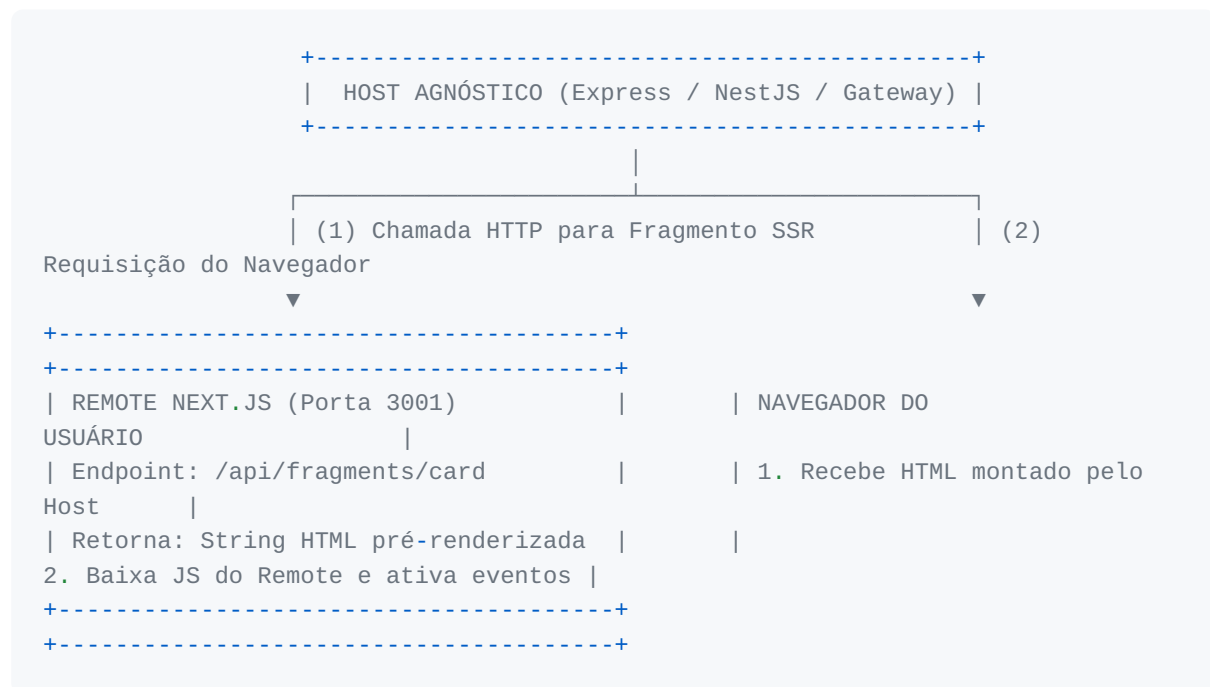
```
const getRemotes = (isServer) => ({
  remote: `remote@http://localhost:3001/_next/static/${isServer ? 'ssr' :
'chunks'}/remoteEntry.js`,
});
```

- No `getServerSideProps`, o Host realiza a busca de dados segura com timeout.

- A renderização utiliza `React.lazy` e `Suspense` protegidos por um `FederatedErrorBoundary`.

5.2. Cenário: Host Agnóstico (Express, NestJS, Remix, Vite, Spring, Go, etc.)

Se a aplicação principal (**Host**) não for construída em Next.js, existem três estratégias consolidadas para renderizar o componente no servidor:



Abordagem A: Fragmentos SSR via HTTP (Recomendada para Produção)

1. O Remote Next.js expõe uma rota que renderiza apenas o componente em formato HTML (ex: `/fragments/card`).
2. O Host agnóstico (Express, NestJS, FastAPI) faz um `fetch('http://remote:3001/fragments/card', { timeout: 800 })` no seu middleware ou controller.
3. O Host injeta a string HTML diretamente no template da página antes de enviar a resposta ao usuário.
4. O Host inclui a tag `<script src="http://remote:3001/_next/static/chunks/remoteEntry.js">` para que o browser hidrate os botões e a interatividade.

Abordagem B: Web Components com Declarative Shadow DOM (DSD)

1. O Remote Next.js emite o componente encapsulado em um Custom Element nativo com `<template shadowrootmode="open">`.
2. O Host agnóstico repassa o Custom Element no HTML da resposta sem precisar ter React instalado.

3. O navegador renderiza o HTML e o CSS instantaneamente no momento da chegada do documento. O script do Web Component conecta os eventos quando terminar de baixar.

Abordagem C: `@module-federation/node` (Para Hosts Node.js não-Next.js)

1. Em servidores Node.js (Express, NestJS, Fastify), instala-se o pacote `@module-federation/node`.
2. O servidor importa o `_next/static/ssr/remoteEntry.js` em tempo de execução, executa `ReactDOMServer.renderToString(React.createElement(RemoteCard, props))` e injeta o resultado na resposta.

6. Validação e Matriz de Testes

Os testes automatizados cobrem todos os estados possíveis da arquitetura:

```
# 1. Instalação e verificação de tipos
pnpm install
pnpm typecheck

# 2. Compilação no Next.js 15
pnpm build

# 3. Execução em produção
pnpm start
```

Resultados dos Testes de Resiliência

```
# Teste de Resiliência e Recuperação Automática
pnpm verify:resilience
```

Saída do Terminal:

```
· Verifying Host Resilience & Graceful Fallback (http://localhost:3000)...
  Host responded with HTTP 200 OK
  Host core page rendered cleanly
  Graceful Fallback Mode active: Host rendered RemoteFallbackCard without
  crashing

  Resilience verification passed! Host survives remote outages seamlessly.
```

Resultados do Teste de SSR (Remote Online)

```
# Teste de Renderização SSR no HTML Inicial
pnpm verify:ssr
```

Saída do Terminal:

```

  - Checking SSR federated component rendering on Host (http://
localhost:3000)...
  -   Asserted: Host Title / Container
      Asserted: Remote Federated Badge
      Asserted: Remote Component Title
      Asserted: Remote SSR Origin Text
      Asserted: SSR Request ID Label

  SUCCESS: Micro-Frontend SSR Federation is fully verified! The remote
component was rendered on the server in the host response.
```

7. Resumo das Lições Aprendidas

1. **Desacoplamento de Dados no SSR:** Nunca confie cegamente no cache de módulos de servidor em Module Federation para dados voláteis de SSR; use endpoints HTTP diretos com timeouts curtos para data fetching isomórfico.
2. **Paridade Rigorosa de Tags DOM:** Evite diferenças de tags entre o componente renderizado e seu respectivo fallback para impedir erros de hidratação no React 18/19.
3. **Pages Router para Module Federation com SSR:** O Pages Router permanece como a base mais sólida e estável para arquiteturas de Micro-Frontends federadas com Next.js.
4. **Isolamento via Error Boundaries:** Todo componente federado deve ser protegido por um Error Boundary para garantir que falhas no runtime do cliente não derrubem a aplicação Host.